

GAGMA

Graph-Augmented GenAI Malware Analyst

An enterprise-grade, Generative AI-powered Android APK malware analysis platform that combines automated reverse engineering, Neo4j call-graph knowledge databases, multi-agent AI behavioral reasoning, and real-time incident response alerting -- purpose-built for the Indian digital banking and financial payments ecosystem.

PROJECT METADATA

Problem Statement: PS1 - GenAI-Based Automated Analysis of Fraudulent APKs

Version: Enterprise Edition v2.0

Architecture: Containerized Microservices (Docker + Caddy Reverse Proxy)

Live Server: <http://3.229.117.157> (AWS EC2 t2.micro, Free Tier)

Repository: <https://github.com/shree-arun/gagma.git>

Frameworks: RBI IT Master Direction 2023, CERT-In SLA, MITRE ATT&CK Mobile

1. Problem Statement & How GAGMA Addresses It

1.1 The Problem (Verbatim from Hackathon Brief)

Fraudsters increasingly distribute malicious mobile applications (APKs) through platforms such as WhatsApp, SMS, email, and phishing links to steal customer credentials, access sensitive information, and perform unauthorized financial transactions. Manual analysis of such APKs is complex, time-consuming, and dependent on skilled cybersecurity experts.

1.2 What the Problem Statement Requires

- A Generative AI-powered malware analysis system for suspicious APK files.
- GenAI for reverse engineering, malware pattern recognition, code interpretation, and threat summarization.
- Static and dynamic analysis: permissions, APIs, embedded code, runtime activities, network comms.
- AI-driven malware pattern detection and threat severity classification.
- Risk score generation with detailed investigation reports and actionable recommendations.
- Faster identification of fraudulent apps for proactive bank cybersecurity.

1.3 How GAGMA Fulfils Every Requirement

GenAI-Powered Analysis	Google Gemini 2.0 Flash performs autonomous code audits, de-obfuscation, and natural-language threat summarization on decompiled APK source.
Reverse Engineering	Androguard library decompiles DEX bytecode, extracts AndroidManifest.xml permissions, recovers Java class hierarchies and method call-graphs.
Static Analysis	Extracts 60+ dangerous permissions, suspicious API calls (SmsManager, Cipher, WebView, ContentResolver), hardcoded URLs, C2 IPs, and obfuscated strings.
Dynamic Analysis	Emulated sandbox analysis checks runtime behaviors: network callbacks, file I/O, crypto usage, and SMS interception patterns.
Pattern Recognition	Multi-agent behavioral engine detects 9+ threat patterns: Banking Trojan, Spyware, Ransomware, SMS Fraud, Phishing, Keylogging, Overlay Attacks, C2 Comms, Data Exfiltration.
Risk Scoring	Weighted 0-100 composite score across 4 dimensions: Permissions (25), APIs (25), Behavioral Patterns (30), Threat Intelligence (20).
Investigation Reports	Auto-generated Markdown reports with MITRE ATT&CK mappings, kill-chain visualization, banking risk flags, and CERT-In formatted incident logs.
Proactive Prevention	Real-time WhatsApp alerts to CSO phones, enterprise SIEM webhook dispatch, and APK blocklist enforcement API.

2. Technical Architecture & System Design

2.1 High-Level Architecture

GAGMA follows a layered microservice architecture deployed inside Docker containers on AWS EC2. A Caddy v2 reverse proxy handles HTTP routing, while the core FastAPI backend orchestrates analysis pipelines, AI agents, graph databases, and alerting services.

Architecture Flow: User Browser --> Caddy Proxy (Port 80) --> FastAPI Backend (Port 8001) --> Analysis Pipeline --> Neo4j Graph DB + GenAI Agents --> Results + Alerts

2.2 Complete Tech Stack

Backend Framework	FastAPI 0.115 with Uvicorn ASGI (2 workers, async I/O)
Language	Python 3.11 (slim Docker base image)
APK Decompilation	Androguard 4.1.2 (DEX disassembly, manifest parsing, call-graph extraction)
Graph Database	Neo4j 5.28 / AuraDB (Malware Knowledge Graph with Cypher queries)
GenAI Engine	Google Gemini 2.0 Flash (primary), OpenAI GPT-4.1-mini, Groq Llama-3.3-70B (fallbacks)
Threat Intelligence	VirusTotal API (hash reputation), AbuseIPDB API (IP reputation scoring)
Persistent Storage	SQLite 3 via SQLAlchemy 2.0 (analyses, blocklist, audit logs, settings)
Frontend	Vanilla HTML5 + CSS3 + JavaScript (glassmorphic dark-theme SOC dashboard)
Graph Visualization	Vis.js Network library (interactive call-graph and campaign cluster rendering)
Reverse Proxy	Caddy v2 Alpine (auto-HTTPS, gzip compression, security headers)
Containerization	Docker + Docker Compose (multi-service orchestration)
Cloud Hosting	AWS EC2 t2.micro (Free Tier), us-east-1 region
Security	SlowAPI rate limiting (120 req/min), CORS lockdown, security headers middleware
Alerting	WhatsApp via CallMeBot API, Enterprise SIEM/MDM JSON webhooks

3. How It Works - Analysis Pipeline

Step 1: APK Upload & Validation

The user uploads a suspicious .apk file via the web dashboard drag-and-drop zone. The backend validates file type, enforces a 50MB size limit, computes SHA-256 hash, assigns a unique analysis ID, and saves the binary to persistent storage.

Step 2: Automated Decompile (Static Analysis)

Androguard disassembles the compiled DEX bytecode into readable class structures. The engine extracts: AndroidManifest.xml permissions (classified as dangerous/normal), suspicious API call patterns (SmsManager, Cipher, WebView, HttpURLConnection), hardcoded URLs and IP addresses, package metadata, and method-level call-graph edges. A performance cap of 30,000 call-graph edges prevents runaway scans on large APKs.

Step 3: Neo4j Knowledge Graph Construction

Extracted components are ingested into a Neo4j Malware Knowledge Graph (MKG). Nodes represent the APK, permissions, suspicious APIs, URLs, and IPs. Edges map structural relationships (REQUESTS, CALLS, CONTACTS, CONNECTS_TO). This enables Cypher-based pattern queries like: 'Show all methods that read SMS and connect to external IPs' -- providing explainable, auditable threat evidence.

Step 4: Multi-Agent GenAI Analysis (Parallel)

Three specialized AI agents execute concurrently: (a) Behavior Agent -- matches 9+ threat patterns against MITRE ATT&CK Mobile matrix, detects banking trojan indicators, generates kill-chain attack sequences. (b) Code Analysis Agent -- queries the Neo4j graph to audit suspicious code blocks, explain obfuscated logic, and identify credential-stealing routines. (c) Threat Intel Agent -- queries VirusTotal for SHA-256 hash reputation and AbuseIPDB for extracted IP abuse confidence scores.

Step 5: Dynamic Analysis Emulation

A sandboxed emulation layer simulates runtime behaviors: network callback triggers, file I/O operations, cryptographic API usage, SMS interception hooks, and overlay attack surface analysis. Results are combined with static findings for comprehensive coverage.

Step 6: Risk Scoring Engine

A weighted composite scoring algorithm produces a 0-100 risk score across four dimensions: Permissions (0-25 pts), Suspicious APIs (0-25 pts), Behavioral Patterns (0-30 pts), and Threat Intelligence (0-20 pts). Final classification: LOW (0-24), MEDIUM (25-49), HIGH (50-74), CRITICAL (75-100). Special weighting boosts scores for high-risk permission combos like SEND_SMS + BIND_DEVICE_ADMIN.

Step 7: Report Generation & Alerting

The system auto-generates a detailed Markdown investigation report containing: executive summary, permission audit table, suspicious API catalogue, behavioral findings with MITRE ATT&CK mappings, banking sector risk flags, attack kill-chain timeline, threat intelligence results, and actionable remediation recommendations. Simultaneously, WhatsApp alerts are dispatched to the CSO's phone, and SIEM webhook payloads are streamed to the bank's Security Operations Center.

4. How GAGMA Helps the Banking Sector

4.1 The Banking Threat Landscape

India's digital banking ecosystem processes billions of UPI transactions monthly. Fraudsters exploit this by distributing rogue APKs disguised as tax filing apps, KYC update tools, loan approval portals, and fake banking apps. These trojans intercept SMS OTPs, overlay spoofed login screens on legitimate apps like SBI YONO, HDFC MobileBanking, Google Pay, and PhonePe, and silently exfiltrate credentials to remote C2 servers.

4.2 Direct Value to Banks

Fraud Prevention (ROI)	By detecting and blocklisting rogue APKs before they reach customers, GAGMA prevents credential theft at the source. This eliminates unauthorized transactions, reducing fraud reimbursement costs that can run into crores annually.
Regulatory Compliance	RBI's IT Master Direction (2023) mandates continuous cyber risk assessment. GAGMA automates this with real-time APK threat scoring, satisfying audit requirements without manual overhead.
CERT-In SLA Adherence	CERT-In requires incident disclosure within 6 hours for severe cyber events. GAGMA generates CERT-In formatted incident logs in under 60 seconds, giving banks a 350x time advantage.
SOC Team Efficiency	Manual APK reverse engineering takes 24-48 hours per sample. GAGMA completes full analysis in under 60 seconds, allowing SOC teams to process 1000x more samples with existing headcount.
Explainable AI Audits	Unlike black-box scanners, GAGMA's Neo4j call-graphs provide visual, queryable evidence of malicious behavior. Auditors can trace exact permission-to-API-to-network chains, satisfying regulatory explainability requirements.
Customer Protection	Automated WhatsApp alerts and SIEM webhooks enable banks to push real-time warnings to customers about active fraud campaigns targeting their institution.

4.3 Banking-Specific Detection Capabilities

- UPI overlay attack detection (GPay, PhonePe, Paytm screen spoofing)
- SMS OTP interception pattern recognition (RECEIVE_SMS + network exfiltration)
- Fake banking app identification (package name impersonation of SBI, HDFC, ICICI)
- Accessibility service abuse detection (keylogging via BIND_ACCESSIBILITY_SERVICE)
- Device admin privilege escalation (ransomware and remote wipe capabilities)
- C2 server communication mapping (hardcoded IPs and domain extraction)

5. Regulatory Compliance Matrix

RBI IT Master Direction (2023)

Chapter IV mandates banks to implement continuous cyber risk assessment frameworks. GAGMA automates APK threat scoring and maintains persistent audit logs in SQLite, providing a complete compliance trail for RBI inspections.

CERT-In Incident Response SLA

Under the 2022 directive, organizations must report cyber incidents within 6 hours. GAGMA's automated pipeline produces CERT-In formatted reports in under 60 seconds, with instant multi-channel alerting to designated personnel.

MITRE ATT&CK Mobile Framework

All behavioral findings are mapped to official MITRE ATT&CK for Mobile tactics (TA0027-TA0039) and techniques (T1417, T1411, T1636, etc.), providing standardized threat classification recognized globally.

NPCI Digital Payment Guidelines

GAGMA deploys targeted heuristics that specifically detect overlay layers, SMS interception hooks, and credential-stealing routines aimed at UPI applications and major payment processors.

IT Act 2000, Section 43A

By providing proactive threat detection and maintaining detailed audit logs, GAGMA helps banks demonstrate reasonable security practices as required under the Information Technology Act.

6. Project Structure & Codebase

6.1 Directory Layout

<code>backend/main.py</code>	FastAPI application entry point with lifespan management
<code>backend/config.py</code>	Environment variable loader (Neo4j, LLM keys, VirusTotal)
<code>backend/routers/analysis.py</code>	APK upload, status polling, and full pipeline orchestration
<code>backend/routers/chat.py</code>	Interactive AI analyst chat endpoint (contextual Q&A)
<code>backend/routers/prevention.py</code>	APK blocklist and prevention gateway API
<code>backend/routers/webhooks.py</code>	SIEM webhook and WhatsApp alert configuration
<code>backend/routers/demo.py</code>	Sandbox threat simulation endpoints
<code>backend/services/apk_analyzer.py</code>	Androguard-based static decompilation engine (483 lines)
<code>backend/services/graph_service.py</code>	Neo4j ingestion, Cypher queries, campaign clustering (18K)
<code>backend/services/risk_scorer.py</code>	Weighted 0-100 composite risk scoring algorithm
<code>backend/services/dynamic_analyzer.py</code>	Sandboxed runtime behavior emulation engine
<code>backend/services/report_generator.py</code>	Markdown report generator with MITRE mappings
<code>backend/services/llm_service.py</code>	Multi-provider LLM abstraction (Gemini/OpenAI/Groq)
<code>backend/services/yara_generator.py</code>	Dynamic YARA signature rule generator per sample
<code>backend/services/whatsapp_service.py</code>	CallMeBot WhatsApp alert dispatcher
<code>backend/services/webhook_service.py</code>	Enterprise SIEM/MDM JSON webhook dispatcher
<code>backend/services/database.py</code>	SQLite persistent storage (analyses, blocklist, audit)
<code>backend/agents/behavior_agent.py</code>	MITRE ATT&CK behavioral pattern detector (470 lines)
<code>backend/agents/threat_intel_agent.py</code>	VirusTotal + AbuseIPDB threat enrichment agent
<code>backend/agents/graph_query_agent.py</code>	Neo4j Cypher-based graph querying agent
<code>backend/agents/code_analysis_agent.py</code>	GenAI code audit and de-obfuscation agent
<code>frontend/index.html</code>	SOC Command Center dashboard (glassmorphic dark theme)
<code>frontend/js/app.js</code>	Frontend logic: upload, polling, graph rendering, alerts
<code>frontend/css/style.css</code>	Complete design system with animations and responsive layout
<code>Dockerfile</code>	Python 3.11-slim container with androguard system deps
<code>docker-compose.yml</code>	Multi-service orchestration (FastAPI + Caddy proxy)

7. Deployment & Infrastructure

7.1 Cloud Deployment (AWS)

GAGMA is deployed on an AWS EC2 t2.micro instance (Free Tier eligible) in the us-east-1 region. The deployment uses Docker Compose to orchestrate two containers: the GAGMA FastAPI backend (port 8001) and a Caddy v2 reverse proxy (port 80/443). Persistent data (SQLite database, uploaded APKs) is stored in Docker named volumes. Health checks run every 30 seconds to ensure automatic container recovery.

7.2 Deployment Commands

```
# Clone and deploy
git clone https://github.com/shree-arun/gagma.git
cd gagma
cp .env.example .env # Configure API keys
sudo docker compose up -d --build

# Verify health
curl http://localhost/health
```

7.3 Key Dashboard Features

- Drag-and-drop APK upload with real-time 6-step progress pipeline visualization
- Interactive Neo4j call-graph rendered via Vis.js with color-coded node types
- Campaign clustering graph showing cross-APK similarity patterns
- Risk score gauge with 4-dimensional breakdown (permissions, APIs, behavior, intel)
- Banking sector risk flags with attack kill-chain timeline
- AI Analyst Chat for contextual Q&A about the analyzed sample
- Exportable Markdown/PDF investigation reports
- Dynamic YARA rule generation per analyzed sample
- Threat intelligence panel with VirusTotal and AbuseIPDB enrichment
- Virtual smartphone mockup simulating real-time WhatsApp CSO alerts
- Enterprise SIEM/MDM webhook integration for automated incident dispatch
- APK blocklist management with prevention gateway API
- Sandbox threat simulation console for demo scenarios

GAGMA -- Securing India's Digital Banking Ecosystem with Graph-Augmented GenAI

Live Demo: <http://3.229.117.157> | GitHub: <https://github.com/shree-arun/gagma>